

Rapid event-triggered adaptation of central pattern generator parameters for quadrupedal robot locomotion

Alex Pitoiset¹, Tim N. Nisslbeck¹, Ajith Anil Meera¹ and Wouter M. Kouw¹

Abstract—Central pattern generators produce efficient rhythmic locomotion from a small set of parameters, but a set tuned for one operating condition is no longer optimal once that condition changes, even when a reactive attitude controller keeps the trunk level. We augment a coupled-Hopf-oscillator CPG with an attitude virtual model controller and study online re-tuning of its gait parameters as the operating condition varies. We separate two questions: *when* to change the gait, resolved by a prediction-error trigger that fires on a sustained loss of performance, and *what* to change it to, addressed by a safe active inference agent that selects gait parameters by minimizing an expected free energy over a learned map from parameters to performance, keeping a persistent memory of which gaits have led to a fall. Because the agent screens each candidate against this memory instead of physically trialing it, as the grid search or Bayesian optimization of a black-box optimizer must, it recovers online without such destabilizing trials. On two perturbations of the robot’s own dynamics – a weakened leg actuator and a shifting trunk payload, each recurring through a continuous bout – the agent learns within the bout to stop selecting the gaits that fall. How far this recovery generalizes, and how much the gait parameterization itself caps it, remain open.

I. INTRODUCTION

Deploying legged machines in constantly changing environments demands an adaptability that engineers have long sought by replicating animal morphologies, such as snake-like robots for constrained spaces or quadruped platforms for irregular terrain [9], [4]. These platforms inherit the mechanical advantages of their biological counterparts, but their control systems often do not: the Boston Dynamics quadruped Spot, for instance, relies on centralized, sensor-intensive controllers that are effective yet complex and expensive, limiting wider deployment [4].

Central pattern generators (CPGs) offer a simpler alternative. In vertebrates, these spinal neural circuits generate the periodic, coordinated signals of movements such as walking, freeing higher brain centers to modulate speed and gait through simple inputs while sensory feedback keeps the rhythm coupled to the body and environment [5], [3]. This decentralized architecture reduces command latency and bandwidth and simplifies high-level control [6], making CPG-based controllers less energy intensive, simpler, and more affordable.

A. Related Work

Among CPG models used in robotics, coupled nonlinear oscillators are biologically plausible and computationally efficient [2], [6]. Their mutual inhibition synchronizes the joints into coordinated gaits with smooth transitions between them [2], [12], [11], and their limit-cycle behavior yields self-sustaining oscillations that recover quickly from perturbations [12]. Governed by only a few parameters such as frequency and amplitude, they suit embedded systems with limited resources, yet most implementations still target high-end platforms, leaving their use on compact systems underexplored.

An open-loop CPG generates stable rhythms on flat ground, but rough or sloped terrain induces body rotations that the rhythm alone cannot correct, so feedback is added to close the loop. Contact and load feedback resynchronize the oscillators to the actual footfalls [11], while posture feedback corrects the trunk attitude directly. Virtual model control (VMC) offers an intuitive route to the latter, computing joint actions as if virtual springs and dampers were attached to the body [10]. Ajallooeian et al. augment a quadruped CPG with virtual springs that pull the trunk level, together with a stumbling-correction reflex, and thereby traverse unperceived rough terrain and slopes that defeat an open-loop CPG [1]. We adopt this attitude VMC as a reactive posture layer beneath the CPG (Section II-C) and ask a complementary question: given such a controller, when and how should the CPG’s own gait parameters be re-tuned as the terrain changes.

A complementary line of work tunes gait parameters as black-box optimization, from measured performance rather than a model of the dynamics. Bayesian optimization suits this setting, where each walking trial is expensive: it fits a Gaussian-process surrogate to the parameter-score pairs seen so far and proposes the next parameters by maximizing an acquisition function such as the upper confidence bound. Run *online*, it scores the parameters of the most recent window, augments the surrogate, and proposes the next window’s parameters, refining the gait during walking; a trust region around the current best keeps proposals from jumping to destabilizing gaits [?]. We take this online Bayesian optimization, together with grid search, as the black-box alternatives our model-based agent is contrasted against.

In the following, we contribute

- a prediction-error trigger that decides *when* to re-tune a central pattern generator, firing on a sustained loss of performance, and

*This work was supported by the Eindhoven Artificial Intelligence Systems Institute

¹Department of Electrical Engineering, Eindhoven University of Technology, Netherlands w.m.kouw@tue.nl

- a safe active inference agent that decides *what* to retune to, selecting parameters from a learned model with a persistent memory of failed gaits and thereby avoiding the destabilizing on-robot trials that grid search and Bayesian optimization require.

II. PROBLEM STATEMENT

We first describe the base platform whose motion defines the locomotion objective, and then the controller driving it: a central pattern generator that produces the coordinated leg motion, augmented with an attitude virtual model controller that reactively corrects posture deviations. We then argue that, even with this reactive posture loop, the parameters yielding stable locomotion depend on the *operating condition* – the combined state of the environment and of the robot’s own body – so that a parameter set tuned for one condition degrades once that condition changes, whether the change is external, such as a different terrain, or intrinsic to the robot, such as an added payload or a weakened actuator. This motivates online adaptation of the parameters as a requirement for resuming stable locomotion after any such shift.

A. Base platform and locomotion objective

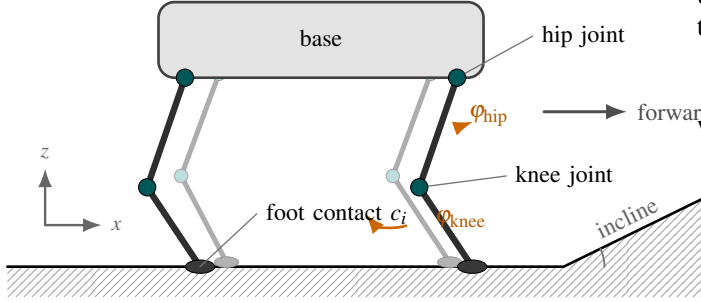


Fig. 1. The Laikago quadruped, its trunk base state ξ , and the twelve leg joints driven by the CPG.

The robot is a floating base actuated through its legs. We describe the trunk by its base state

$$\xi = (p, \Psi, v, \omega), \quad (1)$$

collecting the linear position $p \in \mathbb{R}^3$ and the angular position (orientation) $\Psi \in SO(3)$ of the trunk in a world frame, together with the linear velocity $v = \dot{p}$ and the angular velocity ω . The orientation Ψ is parameterized by the roll, pitch, and yaw (ψ_r, ψ_p, ψ_y) , of which the roll and pitch measure the deviation from an upright attitude.

The legs couple this base state to the joints. For a given motion of the base there is a corresponding configuration of the twelve joints, their rates, and the torques required to realize it,

$$(\phi, \dot{\phi}, \tau) = f(\xi), \quad (2)$$

where ϕ are the joint angles, $\dot{\phi}$ their rates, and τ the joint torques. The map f is unknown: it depends on the full

rigid-body dynamics, the terrain, and the switching contact between the feet and the ground, none of which are available in closed form.

Quadrupedal locomotion is then the problem of finding the joint torques τ such that the base state follows a desired behavior. Concretely, we require the trunk to maintain a target forward velocity v^* while holding an upright angular position, that is roll and pitch near zero,

$$\tau^* = \arg \min_{\tau} \|v - v^*\| + \|(\psi_r, \psi_p)\|. \quad (3)$$

Because f is unknown, these torques cannot be solved for from a model directly. Instead, a central pattern generator produces rhythmic joint-angle references, an attitude virtual model controller adjusts these references to counter posture deviations, and joint-level position controllers track the result through torques, shaping the base motion toward the objective. We introduce both components next.

B. Central Pattern Generator

The CPGs are modeled as a network of four mutually coupled Hopf oscillators, each driving one limb: left front (LF), right front (RF), left hind (LH), and right hind (RH). Coordination between limbs arises from inhibitory coupling through a gait-specific coupling matrix $K = [k_{ij}]$. Let ω_i be the balance between stance and swing frequencies, operationalized as

$$\omega_i = \omega_{\text{stance}} \zeta(-by_i) + \omega_{\text{swing}} \zeta(by_i), \quad (4)$$

with $\zeta(\cdot)$ a sigmoid function. Then, the oscillators evolve as

$$\dot{x}_i = \alpha(u - r_i^2)x_i - \omega_i y_i, \quad (5)$$

$$\dot{y}_i = \beta(u - r_i^2)y_i + \omega_i x_i + \gamma \sum_{j=1}^4 k_{ij} y_j + s_i, \quad (6)$$

where $r_i = \sqrt{x_i^2 + y_i^2}$ is the instantaneous amplitude, u defines the squared amplitude of the limit cycle, and $\alpha, \beta > 0$ are convergence rates (typically $\beta \gg \alpha$ to enforce stronger convergence along the y -axis). The term ω_i of Eq. 4 transitions between the stance and swing frequencies depending on the phase y_i : the sigmoid ζ smoothly interpolates between the two frequencies according to the sign and magnitude of y_i , with the parameter b setting the sharpness of this transition. The coupling gain γ strengthens the coordination between the limbs and reduces the dependence on initial conditions; the coupling matrix $K = [k_{ij}]$ is gait-specific.

The term s_i closes a feedback loop on foot-contact timing [11]. Leg i is classified as swinging when y_i exceeds a small positive threshold and as in stance below a small negative one, with a hysteresis band in between. Comparing this phase against the binary contact state of foot i yields

$$s_i = \begin{cases} K_{\text{stop}}(\omega_i x_i - \gamma \sum_j k_{ij} y_j) & \text{contact matches phase} \\ F_{\text{fast}} \text{sign}(y_i) & \text{otherwise} \end{cases} \quad (7)$$

and $s_i = 0$ inside the hysteresis band. The contact matches the phase when the foot is airborne during swing or loaded during stance; it contradicts the phase upon an early touchdown

or an unexpected loss of ground contact. The gains K_{stop} and F_{fast} set how strongly the sensed footfalls reshape the nominal rhythm, synchronizing the oscillators to the actual contact events.

The oscillator states map to joint-angle references,

$$\varphi_i^{\text{hip}} = \varphi_0^{\text{hip}} + A_{\text{hip}} x_i, \quad \varphi_i^{\text{knee}} = \varphi_0^{\text{knee}} - A_{\text{knee}} \max(0, y_i) + \Delta_i, \quad (8)$$

where $\varphi_0^{\text{hip}}, \varphi_0^{\text{knee}}$ are the joint offsets of the standing pose and $A_{\text{hip}}, A_{\text{knee}}$ are the swing amplitudes. During swing ($y_i > 0$) the knee flexes to lift the foot while the hip advances with x_i ; during stance the knee holds its nominal extension. The term Δ_i is the posture correction supplied by the virtual model controller of Section II-C, and is zero for the open-loop CPG. Joint-level position controllers track these references, producing the joint torques τ of Eq. 2.

The behavior of the CPG is governed by a small set of parameters, collected in the vector

$$\theta = [\gamma, \omega_{\text{swing}}, \omega_{\text{stance}}, F_{\text{fast}}, K_{\text{stop}}, A_{\text{hip}}, A_{\text{knee}}, b], \quad (9)$$

comprising the coupling gain, the phase-dependent stance and swing frequencies, the fast-contact gain F_{fast} and stop gain K_{stop} of the contact feedback (Eq. 7), the hip and knee amplitudes of Eq. 8, and the sigmoid sharpness b of Eq. 4. These parameters set the gait shape, and are held fixed during locomotion.

C. Attitude Virtual Model Control

The contact feedback of Eq. 7 synchronizes the rhythm to the footfalls, but no part of the controller so far senses the trunk attitude. Yet roll and pitch deviations are precisely what drive the robot toward a fall (Eq. 3): on uneven or sloped ground the rhythmic pattern induces unwanted body rotations that it cannot itself correct. Virtual Model Control (VMC) addresses this by imagining virtual components, such as springs and dampers, attached to the robot and computing the joint actions that would arise if these components physically existed [10]. Ajallooeian et al. [1] augment a quadruped CPG with virtual springs spanning a trunk-fixed plane and a horizontal reference plane, so that the resulting virtual forces at the stance feet continually pull the trunk level.

We adopt this construction in a kinematic form suited to position-controlled joints. A virtual spring-damper on each attitude angle produces the correction commands

$$c_r = -(\kappa_r^r \psi_r + \kappa_d^r \dot{\psi}_r), \quad (10)$$

$$c_p = -(\kappa_p^p (\psi_p - \bar{\psi}_p) + \kappa_d^p \dot{\psi}_p), \quad (11)$$

where the κ 's are stiffness and damping gains. The roll is regulated toward zero, since straight walking involves no sustained bank and lateral tip-over is the dominant fall mode. The pitch is regulated toward a slowly adapting baseline $\bar{\psi}_p$, an exponential moving average of the recent pitch with a time constant on the order of one second, so that transient rocking is damped while the sustained pitch offset imposed by an incline is tolerated rather than fought. The commands are distributed over the legs as the knee corrections of Eq. 8,

$$\Delta_i = \text{clip}(\sigma_i^F c_p + \sigma_i^L c_r, \pm \delta_{\text{max}}), \quad (12)$$

where $\sigma_i^F = +1$ for the front legs and -1 for the hind legs, $\sigma_i^L = +1$ for the left legs and -1 for the right legs, and δ_{max} bounds the correction. Adjusting a knee angle changes the effective leg length, so Eq. 12 realizes the virtual springs of Ajallooeian et al. [1] through the position-controlled knees: the legs on a sinking side of the trunk extend to push that corner back up, while the legs on the rising side shorten.

The VMC gains are held at fixed, hand-tuned values and are not part of the parameter vector θ . The attitude loop thus acts at a fast timescale, correcting momentary posture deviations within a stride, but it leaves the rhythmic pattern itself untouched. Ajallooeian et al. [1] observe that such momentary corrections lose effectiveness as the terrain becomes more difficult, when the attitude corrections begin to work against the forward progression planned by the CPG. Whether locomotion remains stable therefore still hinges on the gait parameters θ matching the operating condition, which we examine next.

D. Shifts in Parameter Optimality

For a fixed operating condition, the parameter vector θ can be tuned to produce stable locomotion. Let e denote the *operating condition*: the combined state of the environment and of the robot's own body that the gait must accommodate. It collects extrinsic factors, such as the terrain's incline, friction, and micro-geometry, together with intrinsic ones, such as the trunk's mass distribution and the torque capacity of each actuator. Running the controller with parameters θ under condition e , with the attitude feedback of Section II-C active, induces a distribution $p(\xi | \theta, e)$ over the base state, which we score by the locomotion objective of Eq. 3,

$$V(\theta, e) = -\mathbb{E}_{p(\xi | \theta, e)} \left[\frac{1}{v^*} \|v - v^*\| + \frac{1}{\psi_0} \|(\psi_r, \psi_p)\| \right], \quad (13)$$

setting $V = -c_f$ whenever the robot falls. The scale ψ_0 lets the attitude term dominate, encoding that remaining upright is the primary requirement. In practice the expectation is a time average over an evaluation window, computed on low-pass-filtered angles so that the gait's rhythmic rocking is not penalized, with the pitch measured relative to any sustained attitude offset and progress beyond the target speed not rewarded. The best gait for condition e is then the condition-conditioned optimum

$$\theta^*(e) = \arg \max_{\theta} V(\theta, e). \quad (14)$$

The difficulty is that $\theta^*(e)$ is not constant across conditions. Optimizing Eq. 13 under conditions that differ – in the terrain underfoot, in the load the trunk carries, or in the torque an actuator can deliver – yields distinct parameter vectors, and each is most stable under the condition it was tuned for. A change in condition therefore relocates the optimum,

$$\theta^*(e_{\text{old}}) \neq \theta^*(e_{\text{new}}), \quad (15)$$

a *shift in parameter optimality*. Such shifts arise from external causes, as when the robot steps from one surface onto another, and equally from intrinsic ones, as when it picks

up a payload or an actuator loses part of its torque; some conditions drift continuously, as along a natural landscape, while others change abruptly.

Because θ is held fixed, a shift leaves the controller at a now-suboptimal point, and the resulting attitude deviations compound: as the trunk tilts away from balance, recovery demands corrections beyond what the bounded attitude feedback of Eq. 12 can supply, and locomotion can fail before the episode ends. Resuming stable locomotion after a shift therefore requires detecting that the optimum has moved and re-tuning θ toward $\theta^*(e_{\text{new}})$ online, quickly enough to preserve balance.

III. METHODS

A. Safe active inference agent

Rather than model the trunk dynamics, the agent models the *outcome* of a gait directly: it maintains a probabilistic map from the CPG parameters to the performance they achieve under the current operating condition, and selects the next gait by minimizing an expected free energy over that map. The map is a Gaussian-process (GP) surrogate for the criterion of Eq. 13,

$$V(\theta) \sim \mathcal{GP}(0, \kappa(\theta, \theta')), \quad (16)$$

fitted to a memory $\mathcal{D}_k = \{(\theta_j, V_j)\}_{j < k}$ of the gaits tried so far and the post-shift score each obtained, a fall recorded as a floor value $V_j = V_{\text{fall}}$. Conditioning on $\mathcal{D}_k$ gives a Gaussian posterior predictive with mean $\mu(\theta)$ and variance $\sigma^2(\theta)$ at any candidate gait, over observation noise σ_n^2 . To keep the map learnable from the few queries a single bout provides, only the high-leverage subset of the parameters of Eq. 9 is searched, the rest held at the incumbent; the memory persists across recurrences of the shift, so the map fills in as the same condition is met again and no gait is ever supplied in advance.

The agent selects the next gait by minimizing an expected free energy functional [8]. A goal prior $\bar{p}(V) = \mathcal{N}(V | y^*, \tau^2)$ encodes the preferred outcome y^* , an optimistic target performance, with tolerance τ^2 . Under the Gaussian posterior predictive, the expected free energy of a candidate gait is

$$G(\theta) = \underbrace{\frac{(\mu(\theta) - y^*)^2 + \sigma^2(\theta) + \sigma_n^2}{2\tau^2}}_{\text{goal seeking}} - \underbrace{\frac{1}{2} \ln \left(1 + \frac{\sigma^2(\theta)}{\sigma_n^2} \right)}_{\text{information gain}}, \quad (17)$$

and the agent proposes

$$\hat{\theta} = \arg \min_{\theta} G(\theta) \quad (18)$$

over a quasi-random pool of candidates, excluding gaits already in memory. The goal-seeking term pulls the proposal toward gaits the map predicts will perform near y^* ; the information-gain term rewards resolving predictive uncertainty $\sigma^2(\theta)$. Safety is not a separate constraint but a consequence of the goal prior: a gait the memory associates with falling has a posterior mean $\mu(\theta)$ far below y^* , so its

goal-seeking cost is large and it is avoided, while the agent still explores gaits it has not yet ruled out. The tolerance τ^2 may be fixed or made curvature-aware, tightening where the map is sharply peaked [7]. Because the agent screens each candidate against its own memory before committing, it selects a recovery gait without the destabilizing on-robot trials a black-box optimizer needs to score a candidate, the property Section IV turns on.

B. Prediction-error triggering

Re-tuning θ at every step is wasteful and, as Section IV shows, actively destabilizing, so the agent adapts only when the incumbent gait is no longer matched to its operating condition. We detect that event from the controller's own prediction error, without a model of the perturbation and without being told when it occurs.

During nominal walking the agent measures a baseline forward speed $\bar{v}$ and attitude tilt $\bar{\rho}$ over a short warmup window. Thereafter it tracks, on exponentially smoothed signals, the shortfall against this baseline,

$$\epsilon_k = \frac{\max(0, \bar{v} - v_k)}{\bar{v}} + \frac{\max(0, \rho_k - \bar{\rho})}{\rho_0}, \quad (19)$$

a forward-speed deficit plus an excess tilt, each on its own scale (ρ_0 a reference tilt). A shift in the operating condition drives ϵ_k persistently positive, whereas the stride-frequency ripple of steady walking averages out, so we accumulate the shortfall in a one-sided CUSUM statistic

$$S_k = \max(0, S_{k-1} + \epsilon_k - \kappa), \quad (20)$$

in which the slack κ is the per-step error tolerated as noise, and fire an event when S_k exceeds a bound h . Because the CUSUM integrates the deficit over time, it registers the sustained mismatch a perturbation creates while rejecting the momentary innovations that dominate a one-step residual at 100 Hz.

The monitor is armed only after the robot has walked at the baseline speed and attitude for a sustained interval past a grace period, so that the transient following a recovery is never mistaken for a fresh perturbation; it disarms on firing and re-arms once nominal walking resumes. A single change therefore produces one event, and a sequence of changes re-triggers once per transition rather than latching. Symmetrically, adaptation is paused whenever the robot is walking nominally, so re-tuning runs only while performance is measurably degraded.

IV. EXPERIMENTS

We evaluate the safe active inference agent on the Laikago quadruped in PyBullet, on two perturbations of the robot's own dynamics: a weakened leg actuator (Section IV-A) and a shifting trunk payload (Section IV-B). In both, the agent runs with the attitude VMC of Section II-C active and adapts only the high-leverage subset of the parameter vector θ of Eq. 9, within fixed bounds throughout, while adaptation is gated by the prediction-error trigger of Section III-B. Holding the incumbent gait θ^* (flat) without adapting is the

reference against which the agent is measured. Control runs at $\Delta t = 0.01$ s with a target forward velocity of 0.5 m/s; each perturbation event is scored by the criterion of Eq. 13 over a hold window with the ramp-in excluded, and gait switches are ramped in over 0.3 s so the gait never jumps.

A. Adapting to actuator damage

The first scenario degrades the robot’s own actuation. As the robot walks, one hind leg (the right rear) loses torque: the ceiling on its hip and knee position controllers droops from a healthy 60 N m to 22 N m over roughly 0.8 s, a level at which holding the incumbent gait θ^* (flat) tips the robot on most occurrences, yet a re-tuned gait can still limp forward. The bout is non-episodic and runs for 120 s, with the same leg damaged repeatedly: after each event the leg is healed, the robot stood upright if it fell, and the damage recurs 3 to 8 s later. The agent thus meets the same perturbation many times and should stop reselecting the gaits that fell.

We deploy the safe active inference agent of Section III-A, here searching the high-leverage subset $\{\gamma, \omega_{\text{swing}}, K_{\text{stop}}, A_{\text{hip}}, b\}$ of Eq. 9 along which the healthy and damaged optima differ, the rest held at the incumbent. Its memory is seeded only with the fact that the incumbent falls under the damage, and it persists across recurrences within the bout. Detection reuses the trigger family of Section III-B in a prediction-error form: a CUSUM on the forward-speed deficit and excess tilt relative to the pre-damage baseline fires when the accumulation crosses a bound, re-armed once the robot has recovered after each heal.

Over three 120 s bouts the agent encountered 33 damage events at a mean detection latency of 1.19 s with no false alarms. As its memory fills, the fall rate drops from 67% over the first third of each bout to 19% over the last third: within a single bout the agent learns to avoid the gaits it has seen fail. The recovery is only partial, however. The compensating direction under a weak leg is a lower-amplitude, slower swing, but the incumbent already sits near the lower bound of the amplitude parameters, so the best surviving gait merely limps ($V \approx -0.18$, a reduced forward speed at a larger tilt) rather than restoring nominal walking. The gait parameterization therefore caps how much of the damage the agent can recover, a limit of the representation rather than of the agent, which we revisit in the Discussion.

B. Adapting to payload shifts

The second scenario perturbs the robot’s mass distribution rather than its actuation. The Laikago carries an 8 kg payload rigidly attached above its trunk, whose own mass is 10 kg, and at random intervals the payload shifts off the sagittal plane, 0.215 m laterally and 0.20 m to the rear, ramped over 1 s so the change is a sustained bias rather than an impulse. This lateral offset sits at the edge of recoverability: holding the incumbent gait θ^* (flat) the robot falls on the majority of shifts, yet a re-tuned gait can still walk forward under the load, so the perturbation rewards adaptation without being hopeless. As in the damage scenario the bout is non-episodic and runs for 120 s, with the same shift recurring many times:

after each shift the payload recenters and a new one occurs 2 to 8 s later, and when the robot falls it is stood upright and the payload recentered before walking resumes. An agent that remembers earlier outcomes should therefore stop repeating the parameter choices that fell.

We pair the optimality trigger with the safe active inference agent of Section III-A, whose persistent Gaussian-process memory over the parameter space records which choices led to a fall. At each detected shift the agent proposes a gait by minimizing the expected free energy of Eq. 17, favoring parameters whose predicted performance is high while avoiding the regions its memory associates with falling, and here it searches only the high-leverage subset $\{\gamma, \omega_{\text{swing}}, F_{\text{fast}}, K_{\text{stop}}, b\}$ of Eq. 9. The memory persists across recurrences within the trial, so the map fills in as the shift repeats. Detection reuses the trigger family of Section III-B in a prediction-error form: a CUSUM accumulates the forward-speed deficit and attitude tilt relative to the pre-shift baseline and fires when the accumulation crosses a bound, re-arming once the robot has recovered after each recenter.

Over three 120 s bouts the agent encountered 39 shifts and fell on one of them (3%), at a mean detection latency of 0.4 s, whereas holding the incumbent falls on the majority of the same shifts. The recovered gaits walk forward rather than merely bracing in place, and the agent reaches them without the destabilizing on-robot trials that grid search and Bayesian optimization require, because it screens each candidate against its memory before committing to it.

V. DISCUSSION

Our results separate two aspects of online adaptation that are easily conflated: deciding *when* to change the gait and deciding *what* to change it to. The prediction-error trigger resolves the first cheaply, firing on a sustained loss of performance rather than on stride-frequency ripple, once per perturbation and not during nominal walking, and it re-arms to catch a recurring sequence. The second is where the danger lies. Grid search and Bayesian optimization can only judge a candidate gait by walking on it, and on an already-perturbed robot a poor candidate does not merely score badly, it tips the robot over. The safe active inference agent instead screens each candidate against a learned map with an explicit memory of past falls, so it avoids these destabilizing trials, and within a single bout its fall rate drops as the memory fills. It achieves this not by a faster search toward a distant optimum, but by making small, model-guided corrections that avoid the gaits it has seen fail. This holds even though the CPG is already stabilized by the attitude virtual model controller, so the benefit is complementary to the reactive posture loop rather than a substitute for it.

Several limitations qualify these conclusions. The evaluation is in simulation, over two intrinsic perturbations and a modest number of recurrences, and the reference optima are computed offline. More fundamentally, how much adaptation can recover is capped by the gait parameterization itself: under a weakened leg the compensating gait wants amplitudes below the lower bound the incumbent already sits near, so the

recovered gait limps rather than restoring nominal walking, and the memory-based agent makes local, safe corrections rather than travelling to a distant optimum. A richer control representation, or a model expressive enough to reach the distant optimum without destabilizing exploration, is the central open problem. Validating on hardware, and across a wider range of perturbations, are natural next steps.

VI. CONCLUSIONS

We augmented a coupled-Hopf-oscillator CPG with an attitude virtual model controller and studied when and how to re-tune its gait parameters online as the operating condition changes. A prediction-error trigger decides when to adapt from a sustained loss of performance, and a safe active inference agent decides what to change by selecting parameters from a learned model, with a persistent memory of past falls, rather than by physically trialing candidates. On a weakened leg actuator and a shifting trunk payload, the agent learns within a single bout to stop selecting the gaits that fall, recovering without the destabilizing trials grid search and Bayesian optimization require. How far this recovery generalizes, and how much the gait parameterization itself caps it, remain open.

REFERENCES

- [1] Mostafa Ajallooeian, Soha Pouya, Alexander Sproewitz, and Auke J Ijspeert. Central pattern generators augmented with virtual model control for quadruped rough terrain locomotion. In *IEEE International Conference on Robotics and Automation*, pages 3321–3328. IEEE, 2013.
- [2] J. Collins and S. A. Richmond. Hard-wired central pattern generators for quadrupedal locomotion. *Biological Cybernetics*, 71(5):375–385, 1994.
- [3] Michael H. Dickinson, Claire T. Farley, Robert J. Full, M. A. R. Koehl, Rodger Kram, and Steven Lehman. How animals move: An integrative view. *Science*, 288(5463):100–106, 2000.
- [4] Rohit Dwivedula, Sadanand Modak, Aditya Akella, Joydeep Biswas, Daehyeok Kim, and Christopher J. Rossbach. Configbot: Adaptive resource allocation for robot applications in dynamic environments. *arXiv preprint arXiv:2501.10513*, 2025.
- [5] Martyn Goulding. Circuits controlling vertebrate locomotion: moving in a new direction. *Nature Reviews Neuroscience*, 10(7):507–518, 2009.
- [6] Auke Jan Ijspeert. Special issue: Central pattern generators for locomotion control in animals and robots: A review. *Neural Networks*, 21(4):642–653, 2008.
- [7] Wouter M Kouw. Information-seeking polynomial NARX model-predictive control through expected free energy minimization. *IEEE Control Systems Letters*, 2023.
- [8] Ajith Anil Meera and Wouter M Kouw. Curvature-aware expected free energy as an acquisition function for Bayesian optimization. *IEEE Control Systems Letters*, 2026.
- [9] Kristin Y. Pettersen. Snake robots. *Annual Reviews in Control*, 44:19–44, 2017.
- [10] Jerry Pratt, Chee-Meng Chew, Ann Torres, Peter Dilworth, and Gill Pratt. Virtual model control: An intuitive approach for bipedal locomotion. *International Journal of Robotics Research*, 20(2):129–143, 2001.
- [11] Ludovic Righetti and Auke Jan Ijspeert. Pattern generators with sensory feedback for the control of quadruped locomotion. In *IEEE International Conference on Robotics and Automation*, pages 819–824, 2008.
- [12] Alexander Sprowitz, Ludovic Righetti, and Auke Jan Ijspeert. Passive compliant quadruped robot using central pattern generators for locomotion control. In *IEEE RAS & EMBS International Conference on Biomedical Robotics and Biomechatronics*, pages 710–715, 2008.